

Technical Security Brief

Engineering-level companion to the Security & Privacy Posture Statement, written for technical security reviewers — including an explicit gap register

Version: v1.2 **Last updated:** 10 June 2026 **Owner:** Mads Christian Aanesen, CTO

Distribution: External security review

HOW TO READ THIS DOCUMENT

Every claim here is verifiable in the codebase or infrastructure. Where something is missing, it is listed in the gap register (§11) rather than papered over — we would rather name a gap ourselves than have it discovered.

1 System architecture & trust boundaries

Components

- **Next.js application** (Vercel, Fluid Compute / Node) — marketing site, client account area, staff admin app, and all API routes from one codebase.
- **Supabase** (Postgres + Auth + Storage, Germany/EEA) — single system of record for operational (non-EHR) data.
- **React Native client app** (separate repo) — talks to the same Supabase project and the website's API routes.
- **Medalia** (licensed Icelandic EHR) — formal medical records. No API integration yet; data exchange is manual by licensed clinicians.
- **Biody Manager** (France) — raw body-composition data, fetched on demand via a Supabase Edge Function; results are not persisted in Lifeline's database.

PLANNED — IN PROGRESS, NOT YET LIVE

A direct **Medalia ↔ Biody API integration** is being processed. Once operational, the Lifeline-admin-side Biody connection will be retired and measurements will flow straight into the EHR; the client app's on-demand, user-consented Biody fetch remains. Net effect: one fewer health-data path through the Lifeline layer. No code has shipped for this yet.

Trust boundaries

- **Browser/app → API:** Supabase JWT in the Authorization header; every API route re-derives the user server-side — no client-asserted identity.
- **API → database:** two distinct clients. The anon client is subject to RLS; the service-role client bypasses RLS and exists only in server code. A hard convention forbids importing it (or its env var) into any client-side file.

- **Website → Biody edge function:** requires **both** the service-role bearer (platform JWT gate) and an HMAC signature over the request — a leaked service-role key alone cannot forge a call.

2 Authentication & session management

- Supabase Auth (GoTrue): bcrypt-hashed passwords, short-lived JWT access tokens (~1 h) with rotating refresh tokens; sign-out revokes the refresh token.
- **Staff MFA:** TOTP enrolment is mandatory; the admin app refuses access until the session is stepped up to AAL2. Server-side, **every admin mutation endpoint independently re-verifies AAL2** — the check is not UI-only. Exception: the external-counsel role, which only reaches the legal archive (no patient data).
- **Password flows:** double-entry confirmation everywhere a password is set; email confirmation on our own domain via `token_hash + verify0tp` — raw Supabase action links are never emailed.
- **Brute force:** Supabase platform rate limits on auth endpoints; our own sensitive token endpoints add per-IP database-backed limits (e.g. B2B invite verification: 20 attempts/hour/IP) on top of per-invite lockout, with expired/used tokens short-circuited before verification to prevent enumeration and timing leaks.
- **Offboarding:** deactivating the staff row revokes access centrally — RLS policies and API gates check is-active status on every request, so a deactivated account loses admin access on its next request even with a live token.

3 Authorization model

- Postgres Row Level Security on every table holding personal data; API-mediated tables carry an explicit deny-all policy (`FOR ALL USING (false)`) so the only path is the server API.
- Staff checks inside policies go through SECURITY DEFINER helper functions (`is_active_staff()` , `is_admin_staff()`) — never inline subqueries against the staff table (avoids recursive-policy bugs; one audited implementation).
- API-layer gates are role-aware: reads accept any active staff; writes require admin-write roles *and* AAL2. Read-only roles (external lawyer, medical advisor) are blocked from writes at **both** layers — defence in depth, not a single chokepoint.
- Client-facing data access goes through the anon client under RLS; clients only see their own rows (`auth.uid()` scoping).

4 Secrets & key management

- Application secrets (service-role key, HMAC secret, AI provider keys, SMTP) live in Vercel environment variables — encrypted at rest, scoped per environment, never committed. `.env.local` is gitignored; no secrets in the repository.
- The column-encryption key lives in **Supabase Vault**; only SECURITY DEFINER functions can read it. It is not an application env var, so an application-layer compromise does not directly expose it.
- GitHub Actions secrets (DB connection string for backups) are repository secrets with least scope.

KNOWN GAP — ROTATION

No scheduled rotation policy; keys are rotated reactively. Planned: documented rotation runbook + calendar for the service-role key, HMAC secret and Vault key (the pgcrypto design supports re-encryption migration).

5 Rate limiting & abuse protection

- **Platform:** Vercel DDoS mitigation and TLS termination; Supabase limits on auth endpoints.
- **Application (selective, where it matters):** B2B invite verification 20/hour/IP (DB-backed) + logout; per-user daily quotas on AI document parsing (staff exempt); per-user throttle on error ingestion to stop log flooding.
- No anonymous compute-expensive endpoints — public AI endpoints require an authenticated user.

KNOWN GAP

No global WAF rules beyond platform defaults; rate limits are per-endpoint by design. Accepted at current traffic; revisit at public launch.

6 Backups & disaster recovery

- Supabase managed daily snapshots, 7-day rolling window, restorable from the dashboard.
- **Independent nightly logical backup** (GitHub Actions, 03:00 UTC): `pg_dump` of the full public schema + data into a private storage bucket, 14-day retention, manual trigger for pre-migration snapshots — deliberately a second, portable copy outside the managed snapshot system.
- Storage buckets (signed PDFs etc.) are private with per-bucket RLS; covered by provider redundancy.

KNOWN GAP — RESTORE DRILLS

A full timed restore from the nightly dump into a clean project has not yet been performed as a documented drill. RPO is ~24 h worst case; RTO is untested. Planned before public launch.

7 Vulnerability & dependency management

- April 2026: an external code-level security audit was performed; findings were remediated across three sprints (RLS hardening, audit logging, consent system, column encryption, MFA enforcement, DSR workflow) — captured as posture v1.0.
- TypeScript strict mode + ESLint in development; framework and dependencies kept on current major versions.

KNOWN GAPS

No automated SCA/dependency scanning in CI (updates are manual — cheap to fix, planned). No formal third-party penetration test yet; planned once the public launch surface is final. The April audit was code-review based, not adversarial.

8 Logging, monitoring & detection

- **Immutable health-data audit log** (Postgres triggers, admin-only read, 6-year retention) records every INSERT/UPDATE/DELETE on clients, messages, weight logs and body-composition events with actor identity and role.
- **Error telemetry is fully in-house** (Sentry removed 2026-06; no third-party error service):
 - Server: `captureException` / `captureMessage` helpers write directly to `app_errors` ; a `withErrorReporting` wrapper captures unhandled API-route throws; the Next.js

instrumentation `onRequestError` hook captures uncaught request errors.

- Browser: a global error handler posts to a rate-limited ingestion endpoint.
 - PII redaction (kennitala + email regex) runs before storage on **both** paths; cookies, headers and request bodies are never captured. Error data never leaves the EEA database — one fewer subprocessor than a hosted APM.
 - Trade-off, stated plainly: error reporting now shares fate with the primary database (a hosted APM was independent infrastructure). Vercel runtime logs remain an independent secondary signal.
- Triage UI in the admin; daily error-digest email; weekly cron chases overdue access reviews.

KNOWN GAP — DETECTION IS HUMAN-IN-THE-LOOP

Audit log is query-on-demand; error notification latency is up to 24 h (daily digest, no real-time paging); no automated anomaly detection on access patterns. Accepted at current team size (every staff member personally known); revisit as the team grows.

9 Environments, CI/CD & change management

- GitHub → Vercel: `main` auto-deploys to production; feature branches get isolated preview URLs; TLS managed by the platform.
- Database migrations are plain, idempotent SQL files in the repo, applied manually in the Supabase SQL editor — a deliberate human gate on every schema change, with pre-migration snapshots on demand.

KNOWN GAPS

Single production Supabase project — no separate staging database (preview deployments exercise production data paths; mitigated by the manual migration gate). Single-maintainer repository — no enforced branch protection / second reviewer yet; will be enabled when the engineering team grows beyond one.

10 Endpoint & device security

- Staff sign a device & access policy at onboarding (screen lock, no shared devices, no exfiltration to personal storage); admin access additionally requires TOTP MFA, limiting the blast radius of a stolen password.
- No MDM deployed — contractual + MFA controls only. Accepted at current team size; revisit when clinical staffing scales.

11 Gap register — summary & priority

#	GAP	RISK	PLAN
1	No scheduled secret/key rotation	MEDIUM	Rotation runbook + calendar
2	Restore drill never executed	MEDIUM	Timed drill pre-launch
3	No automated dependency scanning	MEDIUM	Enable Dependabot alerts + PRs
4	No formal penetration test	MEDIUM	Commission once launch surface is final
5	No staging database	MEDIUM	Second Supabase project
6	No real-time alerting (daily digest) nor access-anomaly detection	LOW*	Scheduled audit-log reports + paging on fatal errors
7	No branch protection (single dev)	LOW*	Enable with second engineer
8	No MDM	LOW*	Revisit at clinical scale

*Low at current scale (single-digit, personally-known staff; closed pre-launch user base). These re-rate to Medium at public launch.

12 Changelog

VERSION	DATE	CHANGE
v1.2	2026-06-10	Planned Medalia ↔ Biody API integration documented (plan only — no code shipped; admin-side Biody connection to be retired, app-side fetch stays).
v1.1	2026-06-10	Error telemetry rewritten: Sentry fully removed; in-house pipeline documented incl. shared-fate trade-off and up-to-24h notification latency; server-side reporter hardened with kennitala/email redaction; gap #6 broadened.
v1.0	2026-06-10	Initial release for external technical security review: architecture/trust boundaries, authn/z, secrets, rate limiting, backups/DR, vulnerability management, monitoring, CI/CD, device posture, 8-item gap register.

